SuffAI — Deployment Guide

Changes from Monday 19 October (evening) to Tuesday 20 October 2026, deployed to the live project in one pass. Run every command from `~/Downloads/AI_CUP/AI_CUP-main_Updated` (not `AI_CUP_latest`). Appendix A covers deploying everything into a brand-new GCP project with `deploy/bootstrap_new_project.sh`.

What changed

Release	What it does	Needs
1. Unified BigQuery tables, fresh campaigns, AURA New chat (19 Oct)	Customer photos named `{customer_id}_{order_id}_{item_id}.jpg` become rows in BigQuery `commodities`, and the `garment_items` view counts each garment once. Admin can create customers and orders before any photo exists. **New Campaign** and **Create Another Campaign** start a fresh Ads Lab session. AURA gets a New chat button and can move between tabs.	BigQuery migration; BigQuery access for extraction; all three services
2. One consistent dataset (19–20 Oct)	PIN area names, population and buffers come from Gemini with Google Search (`pin_areas`). Each brand, colour, garment and gender has one spelling (`ref_values`). RFM is recomputed from orders. Similarity links photos exactly like extraction. Ads Lab and Admin show one garment count. Admin can create a Style Assistant login. Dashboards pick up new data within about 30 s.	BigQuery migration; BigQuery access for similarity; all three services; three data scripts
3. Private data out of deploys, shared SKU values, one garment count (20 Oct)	Raw customer data and plaintext logins never enter images or Cloud Build uploads. Brand SKUs use the same attribute values as customer garments. BigQuery is the only garment count: Command Center sends each PIN's garments to similarity. Similarity retries failed BigQuery writes instead of dropping them.	Cloud SQL migration 011; all three services
4. One Geo store scorer (20 Oct)	The server scores stores for the Geo map, the Market brief, the market check and AURA. The browser only draws the results, so all four always agree.	Command Center only

What gets deployed

Piece	Action	Notes
BigQuery `suffai_ops`	Two migrations	Unified tables, then consistent dataset (Step 2)
Cloud SQL	Migration 011	Adds columns only; the current similarity keeps working (Step 3)
IAM	BigQuery Data Editor + Job User for the extraction, similarity and Command Center accounts; Vertex AI User for Command Center	Step 4
`suffai-extraction`, `suffai-command-center`, `suffai-similarity`	Redeploy, in this order	Step 5
Data scripts	RFM, PIN areas, reconcile	Run from your machine (Step 6)
Pub/Sub, Cloud Tasks, Scheduler	No change	No new environment variables either

> Every step is safe to re-run. If part of this is already live, run the guide from the top anyway.

Before you start

- Use Cloud Shell, or a machine with `gcloud`, `bq` and Docker. `deploy_command_center.sh` and `deploy_similarity.sh` build with local Docker, and Cloud Run only runs `linux/amd64` images (an Apple Silicon Mac builds `arm64` by default).
- For Step 3 you need `cloud-sql-proxy` and `psql`, or Cloud SQL Studio in the Console.
- For Step 6 you need Python with `pip install -r command_center/requirements.txt` and Application Default Credentials: `gcloud auth application-default login`.

Step 0 — Set variables and snapshot the current settings

```
cd ~/Downloads/AI_CUP/AI_CUP-main_Updated
export GOOGLE_CLOUD_PROJECT=YOUR_PROJECT_ID
export GCP_REGION=asia-south1
gcloud config set project "$GOOGLE_CLOUD_PROJECT"
gcloud config set run/region "$GCP_REGION"

# Save every service's settings (env vars, Cloud SQL, account) first
for s in suffai-command-center suffai-similarity suffai-extraction; do
  gcloud run services describe $s --format=export > /tmp/$s-before.yaml
done

# The bucket whose uploads feed extraction
export GCS_IMAGE_BUCKET="${GOOGLE_CLOUD_PROJECT}-suffai-garment-images"
ANNOTATION='run.googleapis.com/cloudsql-instances'
export CLOUD_SQL_INSTANCE=$(gcloud run services describe suffai-command-center \
  --format="value(spec.template.metadata.annotations[$ANNOTATION])")

PROJECT_NUMBER=$(gcloud projects describe "$GOOGLE_CLOUD_PROJECT" \
  --format='value(projectNumber)')
sa_of() {
  local sa
  sa=$(gcloud run services describe "$1" \
    --format='value(spec.template.spec.serviceAccountName)')
  echo "${sa:-${PROJECT_NUMBER}-compute@developer.gserviceaccount.com}"
}
EXTRACT_SA=$(sa_of suffai-extraction)
SIM_SA=$(sa_of suffai-similarity)
CC_SA=$(sa_of suffai-command-center)
echo "$CLOUD_SQL_INSTANCE  $EXTRACT_SA  $SIM_SA  $CC_SA"
```

> Don't export DATABASE_URL in this shell. The deploy scripts use `--update-env-vars`, so the services keep their current value. If you export it, the scripts copy it onto the services.

Step 1 — Check that no private files are uploaded

```
gcloud meta list-files-for-upload \
  | grep -E 'data_offline|plaintext|brand_logins_summary|customers\.csv|\.zip$|\.env$'
# no output: Cloud Build uploads none of the private files
```

`.gcloudignore` (Cloud Build) and `.dockerignore` (local Docker) now exclude `data_offline/`, the sample zips and CSVs, `captured_garments/`, `archive/`, `.env` and the plaintext / summary login CSVs. The Command Center image carries only `login_cred_db.csv` (password hashes, used as a login fallback).

Step 2 — BigQuery migrations

```
./command_center/scripts/migrate_bq_unified.sh "$GOOGLE_CLOUD_PROJECT"
./command_center/scripts/migrate_bq_consistency.sh "$GOOGLE_CLOUD_PROJECT"
```

- **Unified:** adds `source` / `created_at` to customers and orders; adds `item_id`, `gcs_object_key`, `image_status`, `source`, `model_version` and `extracted_at` to commodities; tags existing rows

`legacy`; adds customer / order / item ids to `garment_similarity_observations`; creates the `garment_items` view.

- **Consistency:** creates `pin_areas` and `ref_values`, rewrites brand / colour / garment / gender values to one spelling, and recreates `garment_items` without superseded CSV rows.

> Run both, in this order, every time. Each one rewrites the `garment_items` view, and the second has the final definition.

```
DS="${GOOGLE_CLOUD_PROJECT}.suffai_ops"
bq query --use_legacy_sql=false "SELECT source, image_status, COUNT(*) n
  FROM \`$DS.commodities\` GROUP BY 1, 2"
bq query --use_legacy_sql=false "SELECT kind, COUNT(*) n
  FROM \`$DS.ref_values\` GROUP BY kind"
bq query --use_legacy_sql=false "SELECT COUNTIF(is_repeat) repeats, COUNT(*) items
  FROM \`$DS.garment_items\`"
```

> **Don't reload the CSVs after this.** `load_to_bq.sh` replaces whole tables, so it refuses any table that holds Admin-created or photo-pipeline rows. `FORCE=1` overrides that and deletes those rows. Use `APPEND=1` to add only rows with new ids.

Step 3 — Cloud SQL migration 011

Option A, through the Cloud SQL Auth Proxy (keep it running for Step 6):

```
cloud-sql-proxy "$CLOUD_SQL_INSTANCE" --port 5433 &
psql "postgresql://DB_USER:DB_PASSWORD@127.0.0.1:5433/DB_NAME" -v ON_ERROR_STOP=1 \
  -f db/migrations/011_similarity_bq_sync.sql
```

Option B: open Cloud SQL Studio in the Console, paste `db/migrations/011_similarity_bq_sync.sql` and run it.

It adds `order_id`, `item_id` and `bq_synced_at` to `garment_similarity_events`, plus an index on events not yet sent to BigQuery. Older events are marked as sent. Check:

```
SELECT column_name FROM information_schema.columns
WHERE table_name = 'garment_similarity_events'
  AND column_name IN ('order_id', 'item_id', 'bq_synced_at');
-- 3 rows
```

Step 4 — IAM

```
for SA in "$EXTRACT_SA" "$SIM_SA" "$CC_SA"; do
  bq add-iam-policy-binding --member="serviceAccount:${SA}" \
    --role=roles/bigquery.dataEditor "${GOOGLE_CLOUD_PROJECT}:suffai_ops"
  gcloud projects add-iam-policy-binding "$GOOGLE_CLOUD_PROJECT" --condition=None \
    --member="serviceAccount:${SA}" --role=roles/bigquery.jobUser
done
gcloud projects add-iam-policy-binding "$GOOGLE_CLOUD_PROJECT" --condition=None \
  --member="serviceAccount:${CC_SA}" --role=roles/aiplatform.user
```

Account	Why
extraction	Looks up the order, supersedes its CSV items and `MERGE`s the photo's item into `commodities`
similarity	Looks up the photo's owner (orders joined to customers) and inserts the observation
Command Center	Admin creates customers, orders and Style Assistant logins; writes `pin_areas` and RFM; grounded PIN lookups need Vertex AI

If your `bq` version rejects a dataset binding, grant **BigQuery Data Editor** on `suffai_ops` in the Console (BigQuery → `suffai_ops` → Sharing → Permissions). If all three services use the default Compute account, the loop grants the same account three times, which is harmless.

Step 5 — Deploy extraction, then Command Center, then similarity

```
./deploy/deploy_extraction.sh             # USE_CLOUD_BUILD=1: build with Cloud Build
./deploy/04_integrate_command_center.sh   # uses the live extraction / similarity URLs
./deploy/deploy_similarity.sh             # also re-points its Pub/Sub push subscription

CC_URL=$(gcloud run services describe suffai-command-center \
  --format='value(status.url)')
curl -sS "$CC_URL/health" | python3 -m json.tool
# use_bq, database_url, extraction_url, similarity_url: true
# admin_unique_garments: "bigquery"
```

Why this order: each deploy keeps the services that are still on the old version working.

- The new extraction returns `item_attributes`, which the current Command Center ignores.
- The new Command Center sends each PIN's garment list (`candidates`), which the current similarity ignores.
- The new similarity requires `candidates` and uses the columns from migration 011.

Don't use `deploy/deploy_cloud_run.sh`: it recreates the retired `suffai-api` and replaces every service's env vars.

Step 6 — Data scripts

Each script is a dry run unless you pass `--apply`. Read the dry-run output first.

```
pip install -r command_center/requirements.txt

# RFM segments from orders (rule rfm-v1)
python3 command_center/scripts/recompute_rfm.py \
  --project "$GOOGLE_CLOUD_PROJECT"
python3 command_center/scripts/recompute_rfm.py \
  --project "$GOOGLE_CLOUD_PROJECT" --apply

# PIN areas: one billed Gemini + Google Search prompt per PIN; saved PINs are skipped
export ENABLE_VERTEX=true GEO_GROUNDING_LOCATION=global GEO_CITY=Bengaluru
python3 command_center/scripts/backfill_pin_areas.py \
  --project "$GOOGLE_CLOUD_PROJECT"
python3 command_center/scripts/backfill_pin_areas.py \
  --project "$GOOGLE_CLOUD_PROJECT" --apply

# Reconcile leftovers (Cloud SQL through the proxy from Step 3)
DATABASE_URL='postgresql://DB_USER:DB_PASSWORD@127.0.0.1:5433/DB_NAME' \
  python3 command_center/scripts/reconcile_data.py \
  --project "$GOOGLE_CLOUD_PROJECT"
DATABASE_URL='postgresql://DB_USER:DB_PASSWORD@127.0.0.1:5433/DB_NAME' \
  python3 command_center/scripts/reconcile_data.py \
  --project "$GOOGLE_CLOUD_PROJECT" --apply

kill %1   # stop the proxy
```

- **Reconcile** deletes orders, items and observations whose customer is missing. It also deactivates those customers' logins, fixes wardrobe PINs, and queues matched photos that never reached BigQuery so similarity resends them. If you deleted customers by hand (as on 19 October for three PINs), their orders appear in the dry run for deletion.
- Pass `DATABASE_URL` on the command line as shown. If you export it, a later deploy script copies the proxy address onto the services.
- **Optional:** `normalize_sku_attributes.py` rewrites stored brand SKU values to the shared spelling (dry run, then `--apply`, with the same `DATABASE_URL`). Ranking already treats old and new spellings as one value. If you run it, run **Extract & Rank** again on open campaigns.

Step 7 — Smoke tests

```
# Admin username and password from your offline credentials file
curl -sS -X POST "$CC_URL/v1/auth/login" -H 'Content-Type: application/json' \
  -d '{"username":"ADMIN_USER","password":"ADMIN_PASSWORD"}' | python3 -m json.tool
export ADMIN_TOKEN=PASTE_TOKEN

curl -sS -H "Authorization: Bearer $ADMIN_TOKEN" \
  "$CC_URL/v1/admin/unique-summary" | python3 -m json.tool
curl -sS -X POST -H "Authorization: Bearer $ADMIN_TOKEN" \
  -H 'Content-Type: application/json' -d '{"filters":{}}' \
  "$CC_URL/v1/geo/scores" | python3 -m json.tool | head -40
./deploy/05_verify_similarity_e2e.sh
```

Where	Action	Expect
Admin → Customers & Orders	Create a customer with a Bengaluru PIN and tick **Create Style Assistant login**	"Customer … created in <area> (segment New)"; username and password shown once
Admin → Customers & Orders	Create a customer with PIN 110001	Blocked: outside Bengaluru; nothing saved
Admin → Customers & Orders	Add an order for 3 clothes	Shows `0 / 3`; the copy button gives `garments/<customer>_<order>_<item_id>.jpg`
GCS	Upload `…_1.jpg` for that order	A `commodities` row with `source=pipeline`, `image_status=extracted`; the order shows `1 / 3` within about 30 s; `SELECT COUNT(*) FROM garment_similarity_events WHERE bq_synced_at IS NULL` stays 0
GCS	Upload a photo for an order that doesn't exist	Extraction and similarity both log `unlinked`; nothing written
Geo	Pick a PIN	Area card with name, population, 1 / 2 / 4 km buffers and sources
Geo (admin)	Change filters and click a few areas	Each dot's decision matches the Market brief's (unless an external market check lowered it)
Geo (brand)	Open Geo	Only the own brand and Competitors, including in `/v1/geo/scores` responses
Ads Lab (brand)	Extract & Rank for a few PINs	"Photographed garments" equals Admin **Unique garments** for the same PINs; SKU values use the shared spelling (`Navy Blue`, `T-Shirt`)
Ads Lab / Campaigns	**New Campaign**; launch 1P; **Create Another Campaign**	A fresh Ads Lab each time; the launched campaign stays on Campaigns, marked simulated
AURA	New chat; then "Should we open a store in 560008?"	Cleared log and a greeting; the same decision as the Geo dot
Similarity logs	During ranking	`compare_pins … candidates=N missing=0`

Map decisions follow the server rule, which is stricter than the old browser rule: footfall under 40 is No. For example, a PIN with brand share 0.25 and footfall 30 moves from Consider to No.

If something fails

Symptom	Fix
Campaign ranking 400 `candidates required`	Similarity went live before Command Center. Deploy Command Center.
Similarity fails every photo with a missing column (`order_id`, `item_id`, `bq_synced_at`)	Migration 011 has not run on that database (Step 3).
Extraction or similarity logs BigQuery `403 Access Denied`	Step 4 for that account. Pub/Sub keeps retrying, so photos catch up once IAM is fixed.
Admin "Create Customer" → "PIN could not be verified right now"	`ENABLE_VERTEX=true` and Vertex AI User on the Command Center account; retry.
Geo shows PIN codes instead of area names	Run `backfill_pin_areas.py --apply`, or open the PIN on Geo.
Campaign ranking 503 "Garment data unavailable"	Command Center can't read `suffai_ops` (Step 4); see its logs.
Command Center logs `extraction response has no item_attributes`	Extraction is older than Command Center; deploy extraction.
Env vars disappeared after a deploy	Something used `--set-env-vars`. Restore them from `/tmp/<service>-before.yaml`.

Rolling back

Cloud Run keeps earlier revisions. List them with `gcloud run revisions list --service=SERVICE`, then send traffic back with `gcloud run services update-traffic SERVICE --to-revisions=REVISION=100`. Roll back in reverse order: similarity, then Command Center, then extraction. An old Command Center with the new similarity breaks ranking. The migrations aren't rolled back: they add tables and columns and rewrite values to one spelling, and the old services keep working on that data.

Appendix A — A new GCP project from scratch

`deploy/bootstrap_new_project.sh` sets up an empty project and deploys everything above, ending with health checks. It is safe to re-run: each step keeps what already exists.

```
# In Cloud Shell (gcloud, bq, python3 and credentials are ready), from the repo root:
GOOGLE_CLOUD_PROJECT=my-new-project ./deploy/bootstrap_new_project.sh

# Let it create the project and link billing as well:
GOOGLE_CLOUD_PROJECT=my-new-project BILLING_ACCOUNT=XXXXXX-XXXXXX-XXXXXX \
  ./deploy/bootstrap_new_project.sh
```

Outside Cloud Shell you need the Google Cloud SDK, python3, curl and openssl, plus `gcloud auth login` and `gcloud auth application-default login`. You don't need Docker, `psql` or the Cloud SQL proxy: Cloud Build builds the images and `gcloud sql import` applies the migrations. The first run takes about 30–40 minutes, mostly Cloud SQL creation and the three builds.

What it does

1. Checks tools, logins and billing. Enables Cloud Run, Cloud Build, Artifact Registry, Compute, IAM, Storage, BigQuery, Cloud SQL Admin, Secret Manager, Pub/Sub, Vertex AI and Cloud Tasks.
2. Starts creating Cloud SQL PostgreSQL `suffai-ops` in the background. Creates the `suffai` Artifact Registry repo, one service account per service (`suffai-cc`, `suffai-extraction`, `suffai-similarity`, `pubsub-push`) and the image and raw buckets. Lets the Compute Engine default account run builds, since new projects build as that account.
3. BigQuery `suffai_ops`: creates the tables, loads the sample CSVs and the hashed logins (only into empty tables), and runs both migrations. Then recomputes RFM, backfills PIN areas and prints a reconcile dry run (it never applies it).
4. Builds the three images with Cloud Build, after checking that no private file would be uploaded.
5. Cloud SQL: creates database and user `suffai` with a random password, stores `DATABASE_URL` in Secret Manager (`suffai-database-url`) and imports migrations 001–011.
6. IAM: gives each runtime account BigQuery Job User + Data Editor, Vertex AI User, Cloud SQL Client and access to the secret. Command Center gets Object Admin on the image bucket and the right to sign URLs as itself; extraction and similarity get Object Viewer.
7. Deploys extraction and similarity (private), lets `pubsub-push` and Command Center call them, and wires uploads: bucket notification → `garment-image-uploaded` → two push subscriptions (300 s ack deadline, retry backoff, dead-letter topic after 5 attempts).
8. Deploys Command Center (public) with the service URLs, sets up the Cloud Tasks queue, checks every `/health` and prints the login URL.

Options

Variable	Default	Meaning
`GCP_REGION`	`asia-south1`	Region for every resource
`GEMINI_MODEL`	`gemini-2.5-flash`	Extraction, chat / AURA and grounded Geo lookups
`SQL_TIER`	`db-g1-small`	Cloud SQL machine (Enterprise edition, PostgreSQL 15)
`LOAD_SAMPLE_DATA`	`1`	Load `command_center_data/sample/*.csv`. These hold customer names, numbers and addresses; `0` starts with empty tables
`BACKFILL_PIN_AREAS`	`1`	One billed grounded prompt per PIN; `0` lets Geo look PINs up when opened
`ENABLE_CLOUD_TASKS`	`1`	Run Ads Lab ranking jobs on Cloud Tasks
`GEO_CITY`	`Bengaluru`	City for PIN checks and store lookups

How the new project differs from the live one

- **`DATABASE_URL` is a Secret Manager secret**, not a plain env var. For later redeploys with `deploy_extraction.sh`, `04_integrate_command_center.sh` and `deploy_similarity.sh`, export only `GOOGLE_CLOUD_PROJECT` and `GCP_REGION`. Exporting `DATABASE_URL` makes the deploy fail, because Cloud Run won't turn a secret into a plain value.
- **Each service has its own account** instead of the default Compute account, so later redeploys keep the account.
- **Command Center runs as one instance** (`--max-instances=1`, CPU always allocated), because login sessions and AURA state live in memory.
- **Extraction uses `GEMINI_MODEL`**. The code default `gemini-2.0-flash` may not be available to new projects.

After it finishes

1. Open the printed `/login` URL and sign in with an account from `login_cred_db.csv`. The passwords are in your offline `login_creds_plaintext.csv`, which is never uploaded.
2. Run the Step 7 smoke tests: create a customer and order in Admin, then upload `garments/<customer_id>_<order_id>_1.jpg` to the printed bucket.
3. Photos that fail five times land in the `garment-image-uploaded-dlq-sub` subscription.
4. To apply the reconcile it printed: `python3 command_center/scripts/reconcile_data.py --project PROJECT --skip-cloud-sql --apply`.

Running cost: Cloud SQL runs all the time, and Command Center is billed while its instance is up. Gemini calls are billed per request. To remove everything: `gcloud projects delete PROJECT`.

Organization policies: if `/health` returns 403, a policy is blocking public (`allUsers`) access to Cloud Run. If Cloud SQL creation fails, a policy may block public IP. Personal accounts without an organization have neither.